Assignment 1 - Time-Series Econometrics

Jasper Sänger

2024-11-26

I. Importing data sets, tidying & hypothesis on patterns

1. *Produce a time plot. Succinctly, describe what you see and formulate hypotheses on the patterns that might be present in the data, based on what you see.*

```r
#loading necessary libraries
library(fpp3)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr
```

```
## -- Attaching packages -------------------------------------------- fpp3 1.0.1 --
```

```
## v tibble      3.2.1     v tsibble     1.1.5
## v dplyr       1.1.4     v tsibbledata 0.4.1
## v tidyr       1.3.1     v feasts      0.4.1
## v lubridate   1.9.3     v fable       0.4.0
## v ggplot2     3.5.1
```

```
## -- Conflicts ------------------------------------------------- fpp3_conflicts --
## x lubridate::date()    masks base::date()
## x dplyr::filter()      masks stats::filter()
## x tsibble::intersect() masks base::intersect()
## x tsibble::interval()  masks lubridate::interval()
## x dplyr::lag()         masks stats::lag()
## x tsibble::setdiff()   masks base::setdiff()
## x tsibble::union()     masks base::union()
```

```r
library(readr)

#loading & viewing data
data_1 <- read_csv("/Users/user/Documents/Master_Study/2nd Trimester/Time Series Econometrics/assignmen
```

```
## Rows: 168 Columns: 2
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (1): <dbl>
## date (1): t
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
data_2 <- read_csv("/Users/user/Documents/Master_Study/2nd Trimester/Time Series Econometrics/assignmen
```

```
## Rows: 1673 Columns: 2
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (1): u
## date (1): t
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
#turning datasets in time series format and renaming columns

dataset_1 <- data_1 %>%
  rename(Date = t, Value = `<dbl>`)
ts_dataset_1 <- dataset_1 %>%
  as_tsibble(index = Date)
glimpse(ts_dataset_1)
```

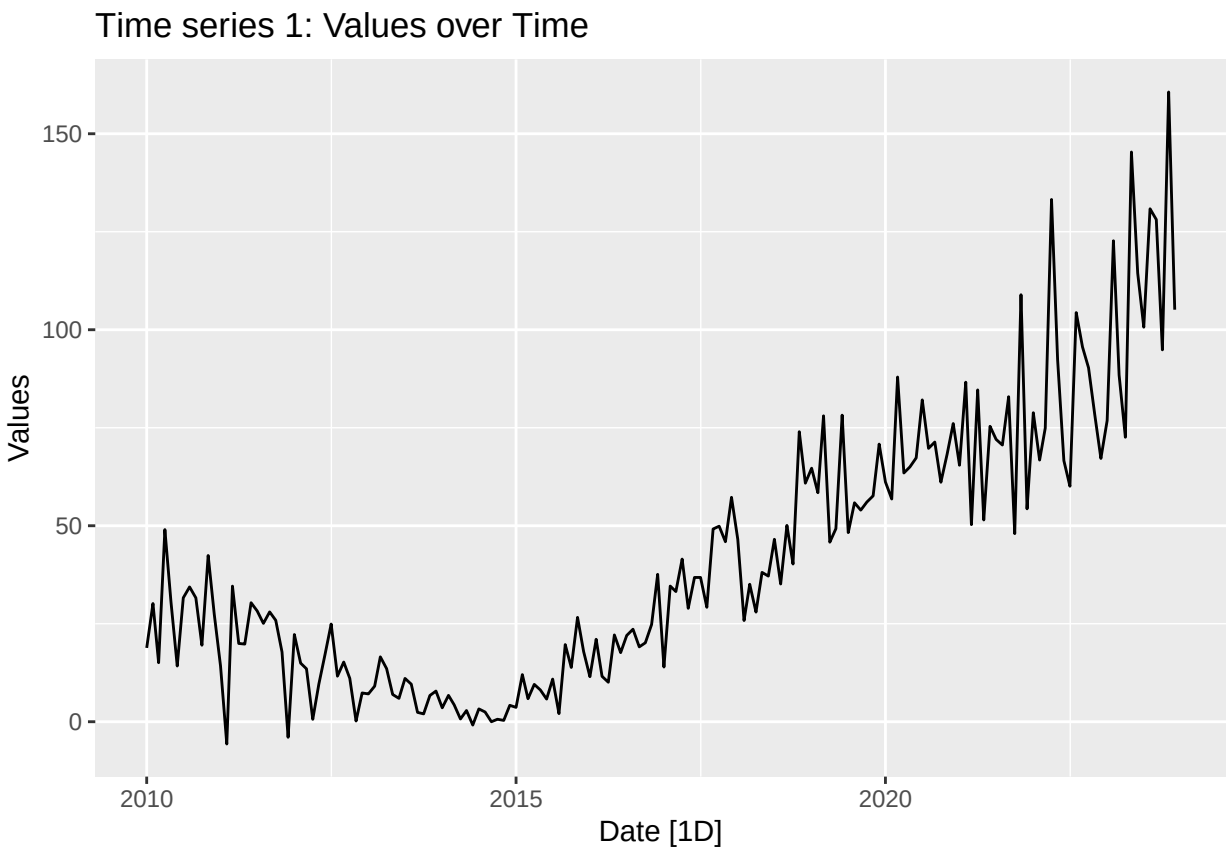
```
## Rows: 168
## Columns: 2
## $ Date  <date> 2010-01-01, 2010-02-01, 2010-03-01, 2010-04-01, 2010-05-01, 201~
## $ Value <dbl> 18.830456, 30.155932, 15.061700, 49.008906, 30.590584, 14.216548~
```

```r
dataset_2 <- data_2 %>%
  rename(Date = t, Value = u)
ts_dataset_2 <- dataset_2 %>%
  as_tsibble(index = Date)
glimpse(ts_dataset_2)
```

```
## Rows: 1,673
## Columns: 2
## $ Date  <date> 2019-01-01, 2019-01-02, 2019-01-03, 2019-01-04, 2019-01-05, 201~
## $ Value <dbl> 26.65683, 27.16499, 26.90875, 28.52006, 28.52006, 28.01191, 28.7~
```

```r
autoplot(ts_dataset_1) +
  labs(y="Values",
       title = "Time series 1: Values over Time")
```

```
## Plot variable not specified, automatically selected `.vars = Value`
```



The time series graph above depicts values plotted over monthly intervals spanning from January 2010 to December 2023. While one can observe a negative trend prior to 2015, values are significantly trending upwards from that point forward.

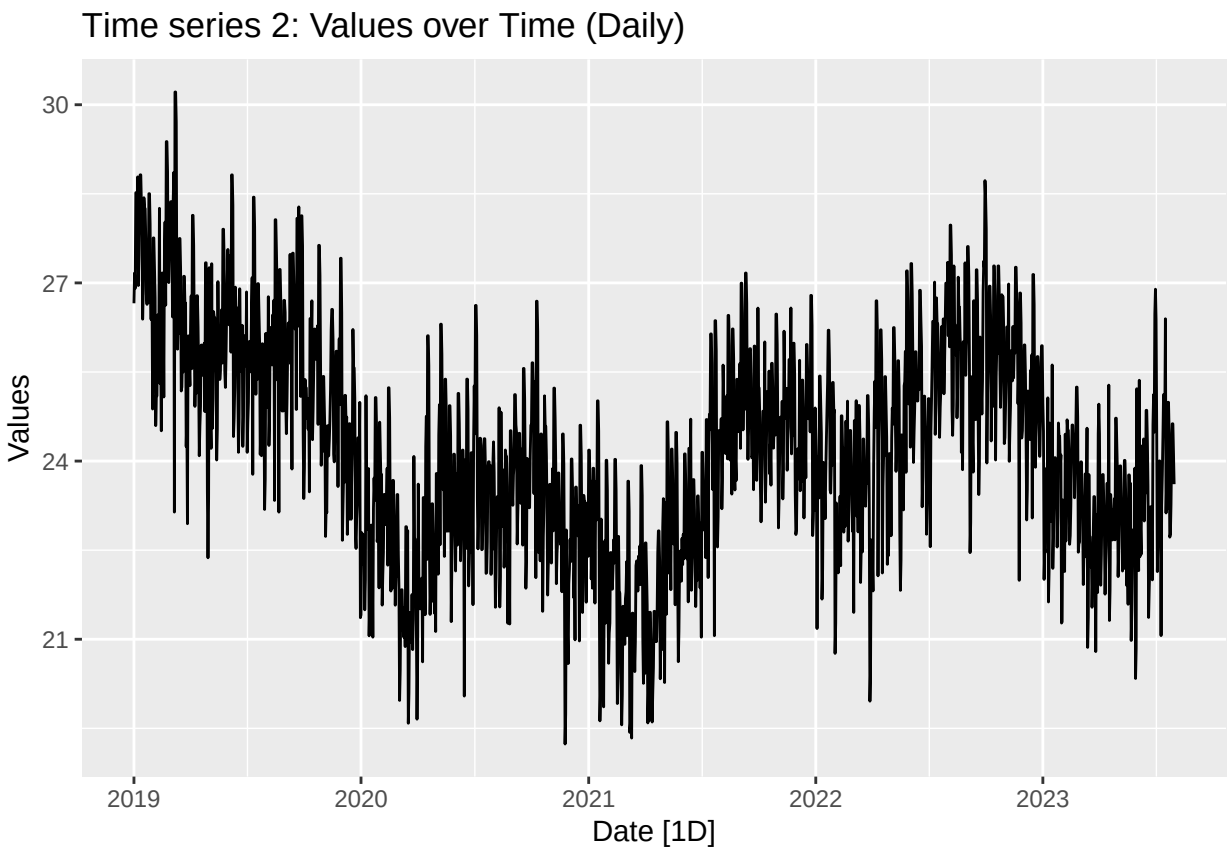
Fluctuation volatility or variability in the monthly data from 2010 to 2013 is higher than in the next 5 years up until approximately 2021. In the final 2 years, one can observe increasingly higher variability again.

There seems to be no indication of seasonality. The fluctuations appear to be irregular and no repeating patterns over consistent monthly or yearly intervals can be spotted.

Potential hypotheses on patterns are; (1) long-term growth trend, (2) volatility in early years (2010-2013) could be a result of a period of instability, (3) structural changes could have occurred around the year 2015, and finally, (4) external disruptions could have caused variability in the last two years of the recorded data.

```
autoplot(ts_dataset_2) +
  labs(y="Values",
       title = "Time series 2: Values over Time (Daily)")
```

```
## Plot variable not specified, automatically selected `.vars = Value`
```



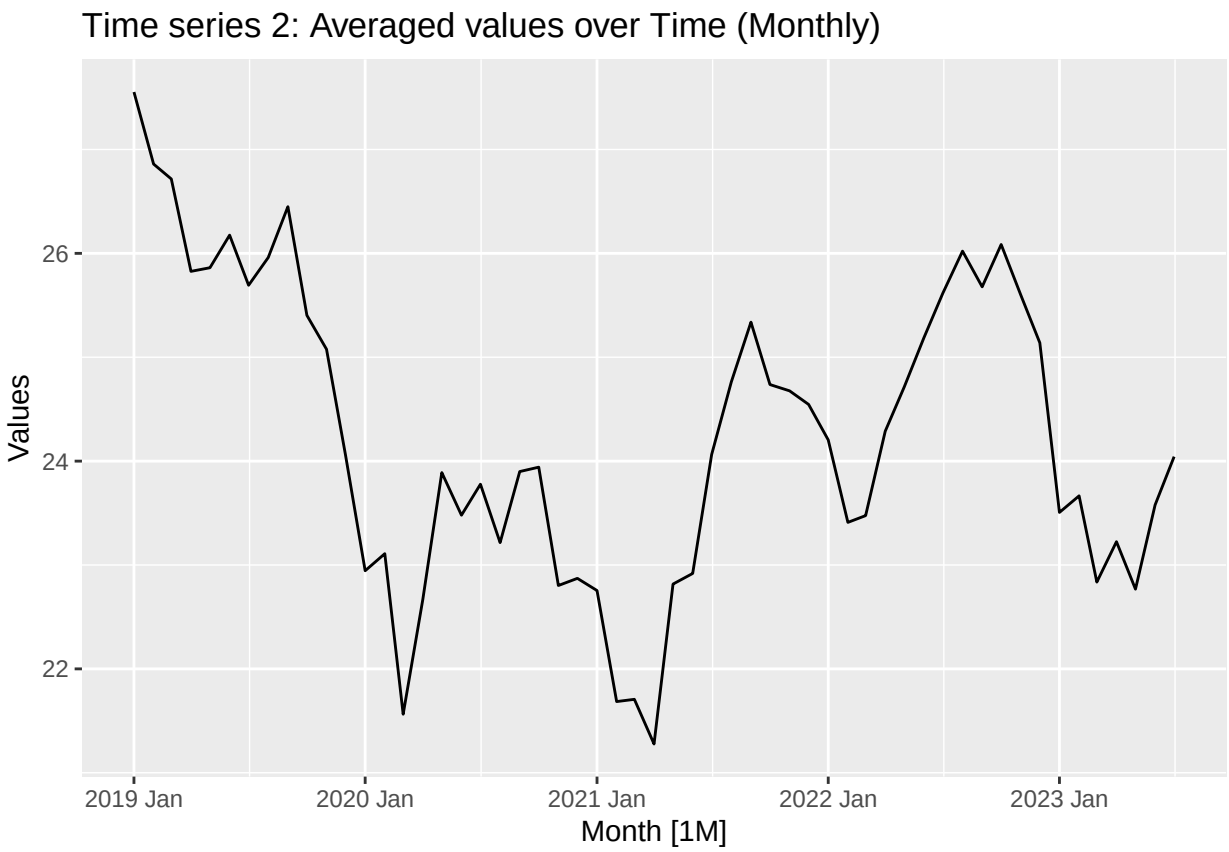
The time series graph above depicts daily values spanning from January 2019 to July 2023. No general trend can be spotted over the 4 years and 7 months. Year 2019 in isolation depicts a downward trend, while the succeeding years aggregated have neither a negative or positive trend. Variability remains fairly constant throughout the entire span of data.

There seems to be potential seasonality within each month (perhaps on a weekly or biweekly level), but this has to be investigated further as it is difficult to decipher by looking at this graph only. A second more likely seasonality can be seen on an approximately 6 month level, where values on average, decline from October to March and rise from April to September. This seasonality is more visible from years 2020 to 2023.

Potential hypotheses on patterns are; (1) seasonal component showcased by the short-term fluctuation may potentially be spikes in daily values at a certain point in each month but could be white noise, (2) second seasonal component showing a decline in values in quarter 4 and quarter 1 and a rise in quarter 2 and 3.

```
ts_dataset_2_monthly <- ts_dataset_2 %>%
  index_by(Month = ~ yearmonth(.)) %>%
  summarise(AverageValue = mean(Value))

autoplot(ts_dataset_2_monthly) +
  labs(y="Values",
       title = "Time series 2: Averaged values over Time (Monthly)")
```

```
## Plot variable not specified, automatically selected `.vars = AverageValue`
```



The graph above depicts data set 2 with values aggregated on a monthly basis to enhance visual interpretation of the yearly seasonality. The patterns and hypothesis addressed in the first graph of data set 2 hold. However, the daily spikes and troughs are no longer visible and cannot be interpreted as a seasonality within each month or subgroup of a month or potentially just as white noise. To interpret the visualizations spanning over the entire data set 2 better, the change of indexes to monthly will be incorporated from here on forward and replace the daily index, wherever required for readability purposes.

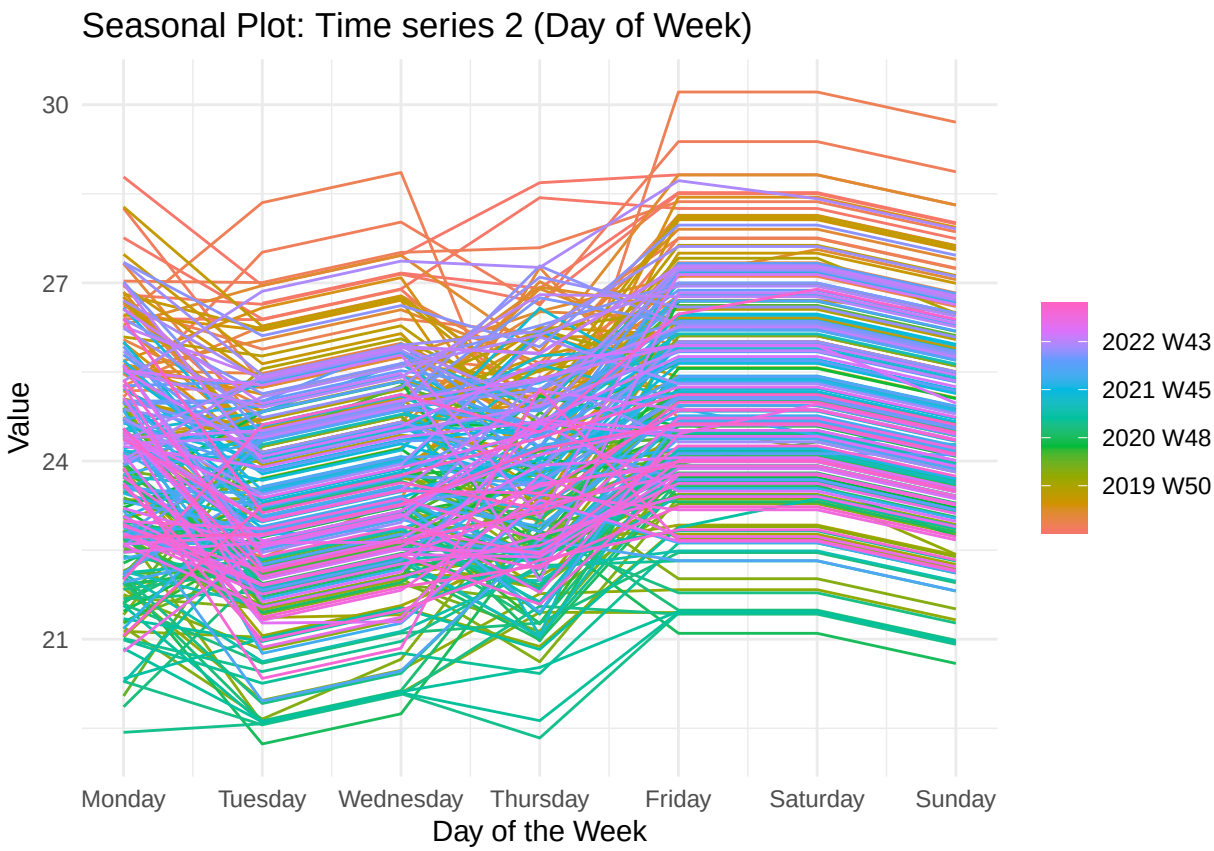
II. Seasonality exploration & hypothesis elaboration

2. *If you find it necessary based on the hypotheses formulated in the previous point, explore the presence of seasonality. Do this for any seasonal period compatible with the series. For each seasonal period you consider, create both the* `gg_season()` *and* `gg_subseries()` *plots. You might need to aggregate the observations to see higher levels of seasonality more clearly. Conclude by succinctly commenting on the evidence provided by these plots, adjusting or expanding the hypotheses formulated above.*

The presence of seasonality was hypothesized for data set 2, potentially within each month and secondly, within each year. Therefore, only data set 2 will be analyzed in this section. Data set 1 does not show any seasonality from the initally inspected graph and will therefore not be scrutinized in part II.

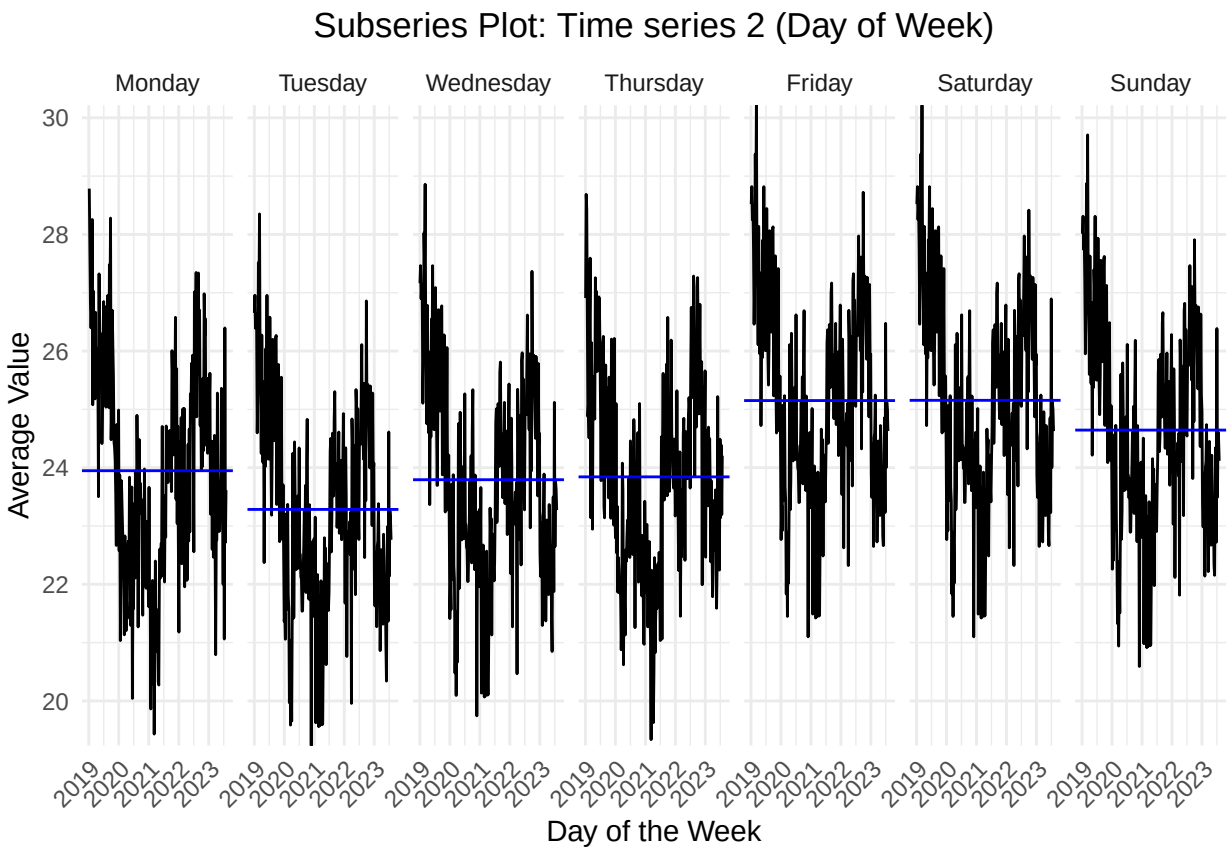
```
gg_season(ts_dataset_2, period = "week") +
  labs(
    y = "Value",
    x = "Day of the Week",
    title = "Seasonal Plot: Time series 2 (Day of Week)"
  ) +
  theme_minimal()
```

```
## Plot variable not specified, automatically selected `y = Value`
```




```r
gg_subseries(ts_dataset_2, period = "week") +
  labs(
    y = "Average Value",
    x = "Day of the Week",
    title = "Subseries Plot: Time series 2 (Day of Week)"
  ) +
  scale_y_continuous(breaks = seq(20, 30, by = 2), expand = c(0, 0)) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(hjust = 0.5)
  )
```

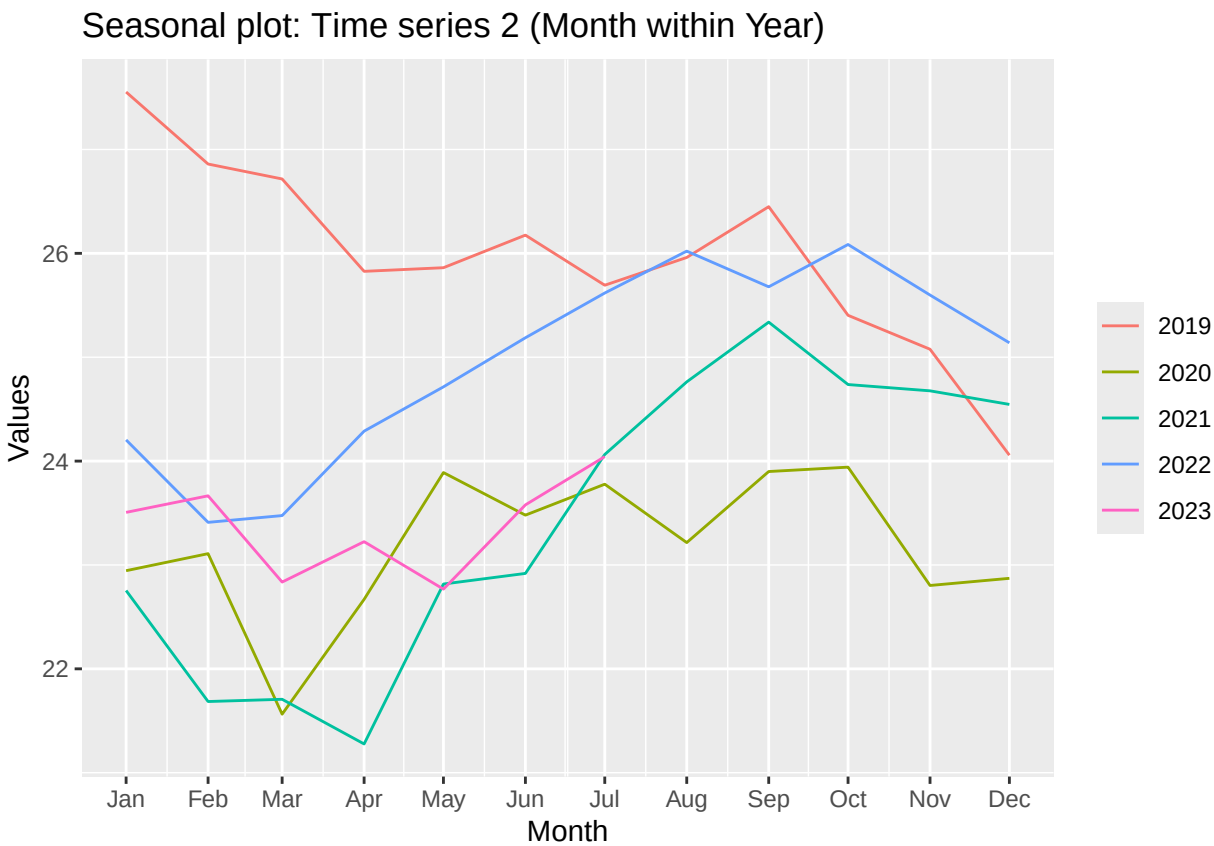
```
## Plot variable not specified, automatically selected `y = Value`
```



Based on the two graphs above, seasonality can indeed be spotted on a weekly basis, where values tend to spike on Fridays and remain higher on the weekend compared to Monday - Thursday. Therefore, this 'short-term' fluctuation is seemingly not white noise as it does follow a seasonal pattern. This will be further investigated in autocorrelation graphs in part III.

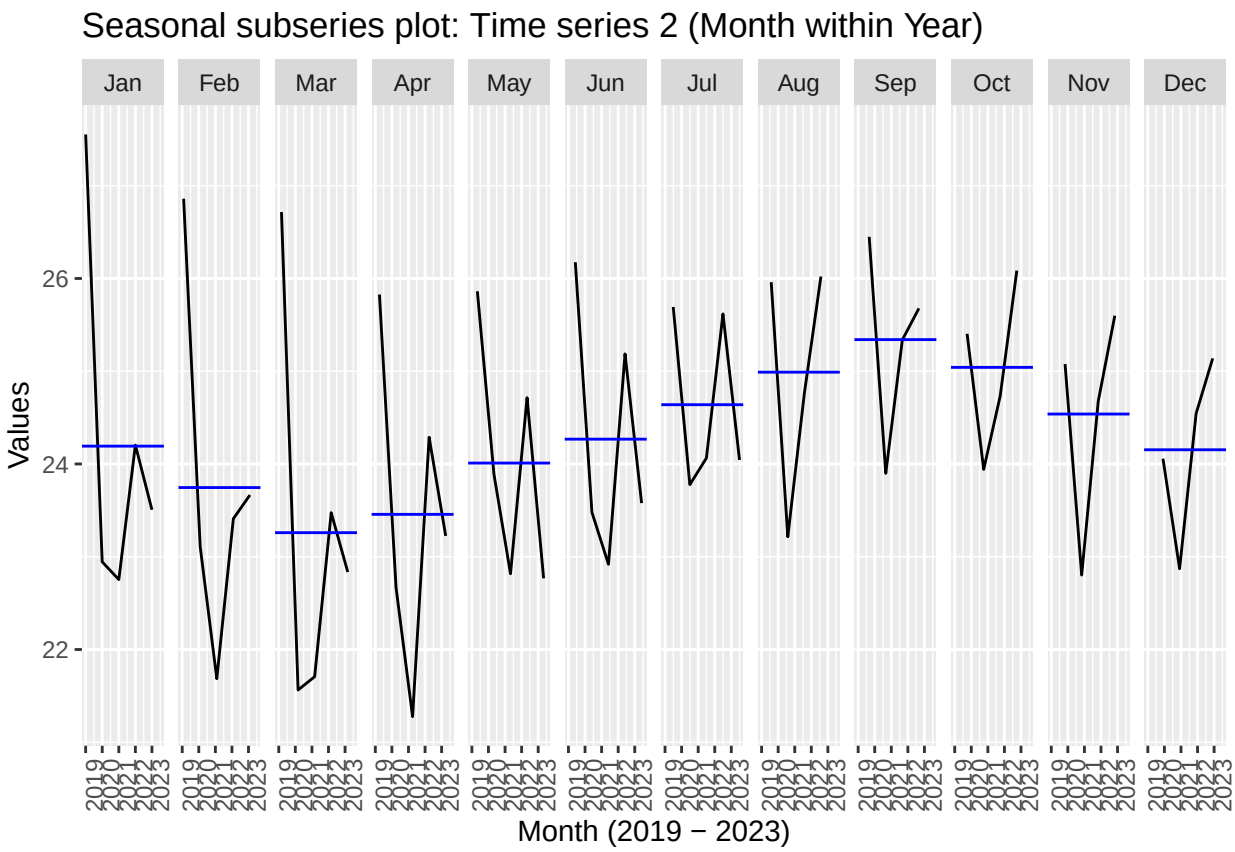
```
gg_season(ts_dataset_2_monthly) +
  labs(y = "Values",
       x = "Month",
      title = "Seasonal plot: Time series 2 (Month within Year)")
```

```
## Plot variable not specified, automatically selected `y = AverageValue`
```




```r
gg_subseries(ts_dataset_2_monthly) +
  labs(y = "Values",
       x = "Month (2019 - 2023)",
      title = "Seasonal subseries plot: Time series 2 (Month within Year)")
```

```
## Plot variable not specified, automatically selected `y = AverageValue`
```



As hypothesized, there is seasonality visible year to year on a monthly basis. Values tend to rise from In April, peak in September and decline until March. This is particularly evident in the sub-series graph by looking at the blue lines indicating the average value over the five years for each month.

III. Visualizing correlation in time series

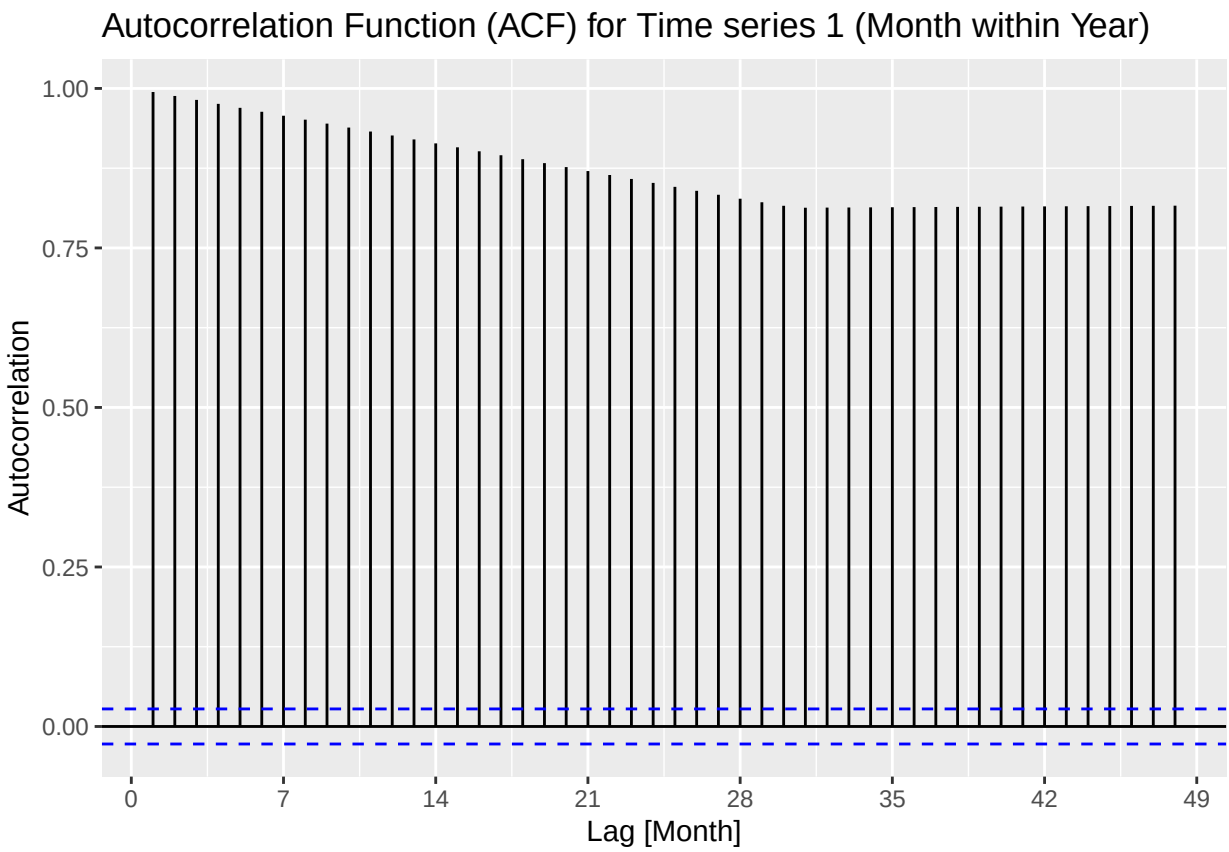
3. *Use the correlogram to probe your hypotheses. You might want to use more than one correlogram, if you have used different levels of aggregation. Succinctly, interpret the insights of the correlogram(s).*

```
# Fill missing dates
ts_dataset_1_fill <- ts_dataset_1 %>%
  fill_gaps()
# Impute missing values by carrying the last observation forward
ts_dataset_1_fill <- ts_dataset_1_fill %>%
  fill(Value, .direction = "downup")
```

To ensure that the time series 1 has a complete sequence of dates, the date gaps are filled by carrying the last observed value forward and the next valid observation backward. Only by doing so, the correlogram can be applied to the time series 1.

```
ACF(ts_dataset_1_fill, lag_max = 48) %>% autoplot() +
  labs(y = "Autocorrelation",
       x = "Lag [Month]",
      title = "Autocorrelation Function (ACF) for Time series 1 (Month within Year)")
```

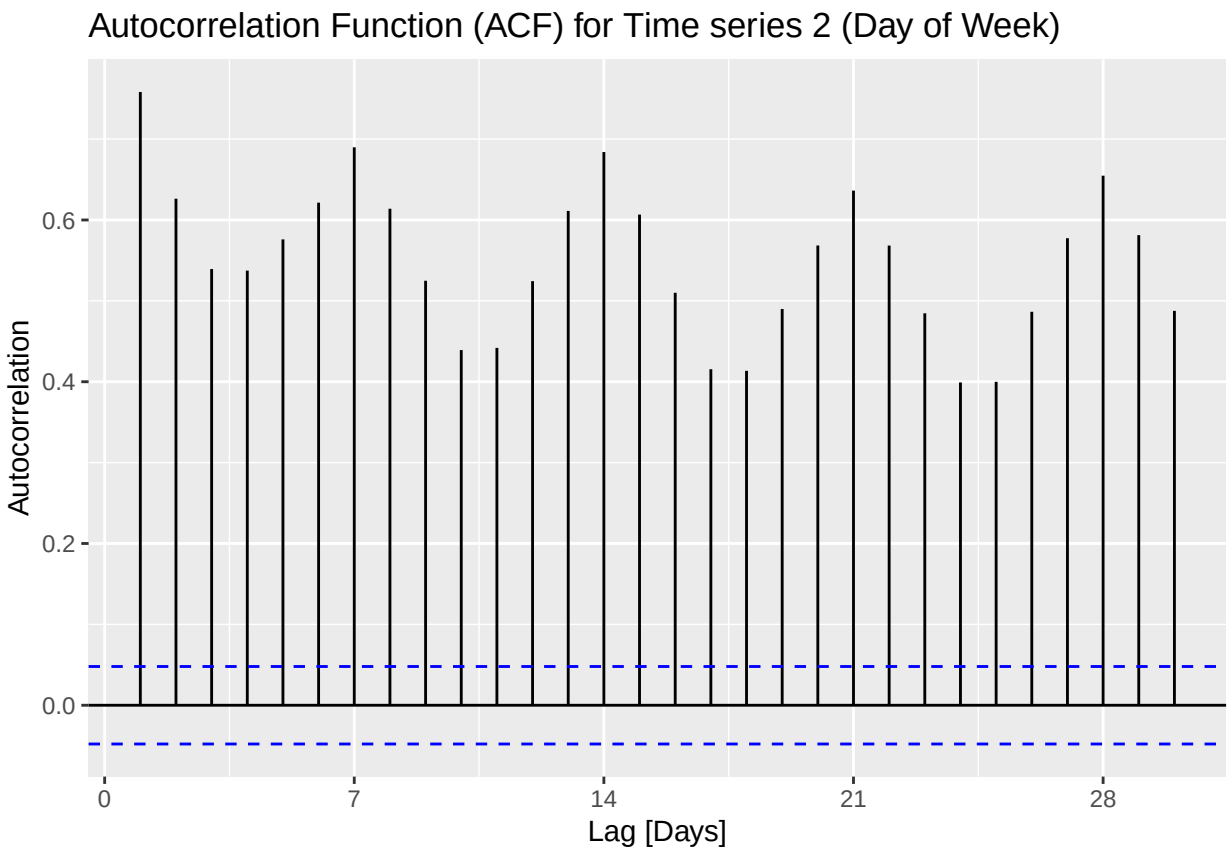
```
## Response variable not specified, automatically selected `var = Value`
```



The correlogram above illustrates consistent and significant positive autocorrelations across all 48 lags. This indicates a strong and persistent dependence in the time series, where past values at a monthly frequency continue to influence future values significantly. The uniform strength of the autocorrelations suggests a lack of seasonal patterns but highlights a high degree of persistence over time.

```
ACF(ts_dataset_2, lag_max = 30) %>% autoplot() +
  labs(y = "Autocorrelation",
       x = "Lag [Days]",
      title = "Autocorrelation Function (ACF) for Time series 2 (Day of Week)")
```

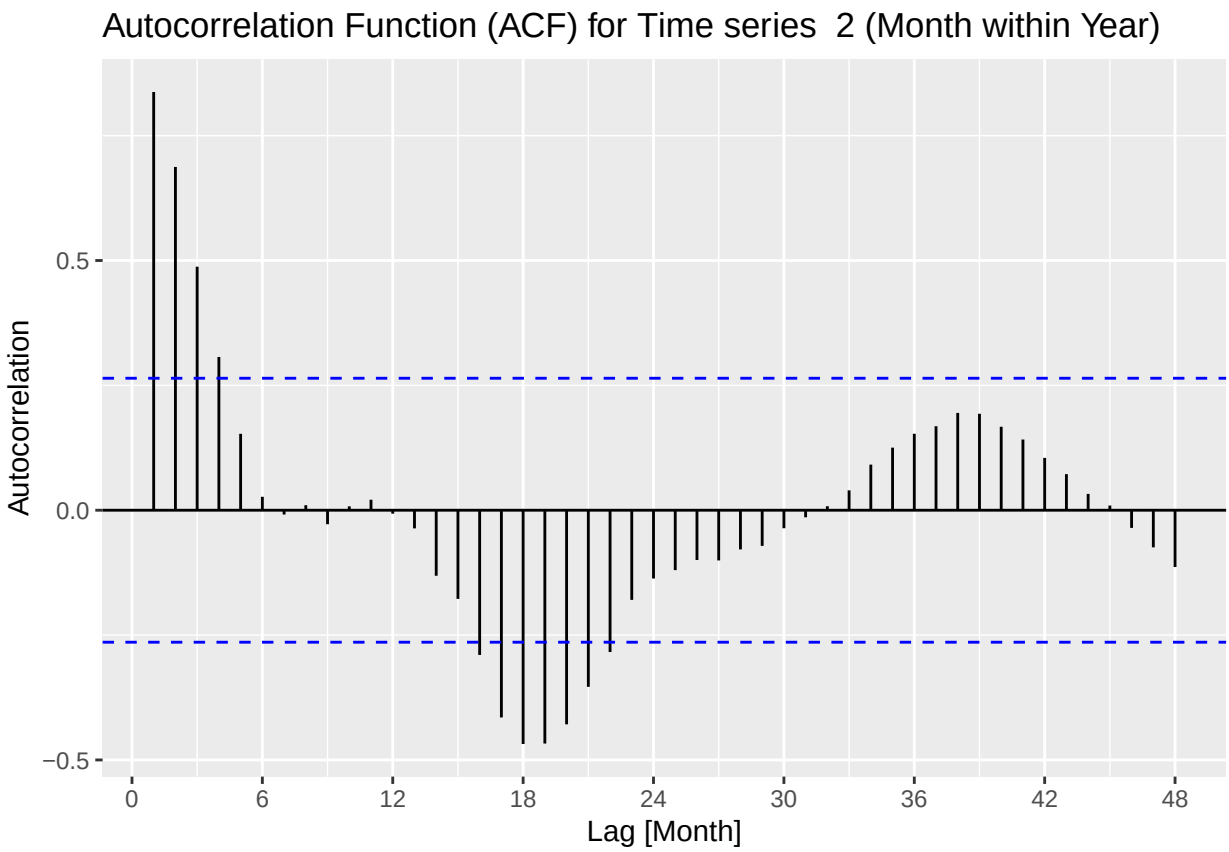
```
## Response variable not specified, automatically selected `var = Value`
```



The correlogram above shows autocorrelations for Dataset 2 at 30 daily lags, revealing a strong and clear weekly seasonality pattern. Significant autocorrelations occur at lags that are multiples of 7 (e.g., 7, 14, 21, and 28 days), consistent with a weekly cycle. The persistence of autocorrelations decaying gradually over time further highlights the presence of a structured seasonal pattern in the data. This consistent behavior implies that past values significantly influence future values, particularly in a weekly context.

```
ACF(ts_dataset_2_monthly, lag_max = 48) %>% autoplot() +
  labs(y = "Autocorrelation",
       x = "Lag [Month]",
      title = "Autocorrelation Function (ACF) for Time series  2 (Month within Year)")
```

```
## Response variable not specified, automatically selected `var = AverageValue`
```



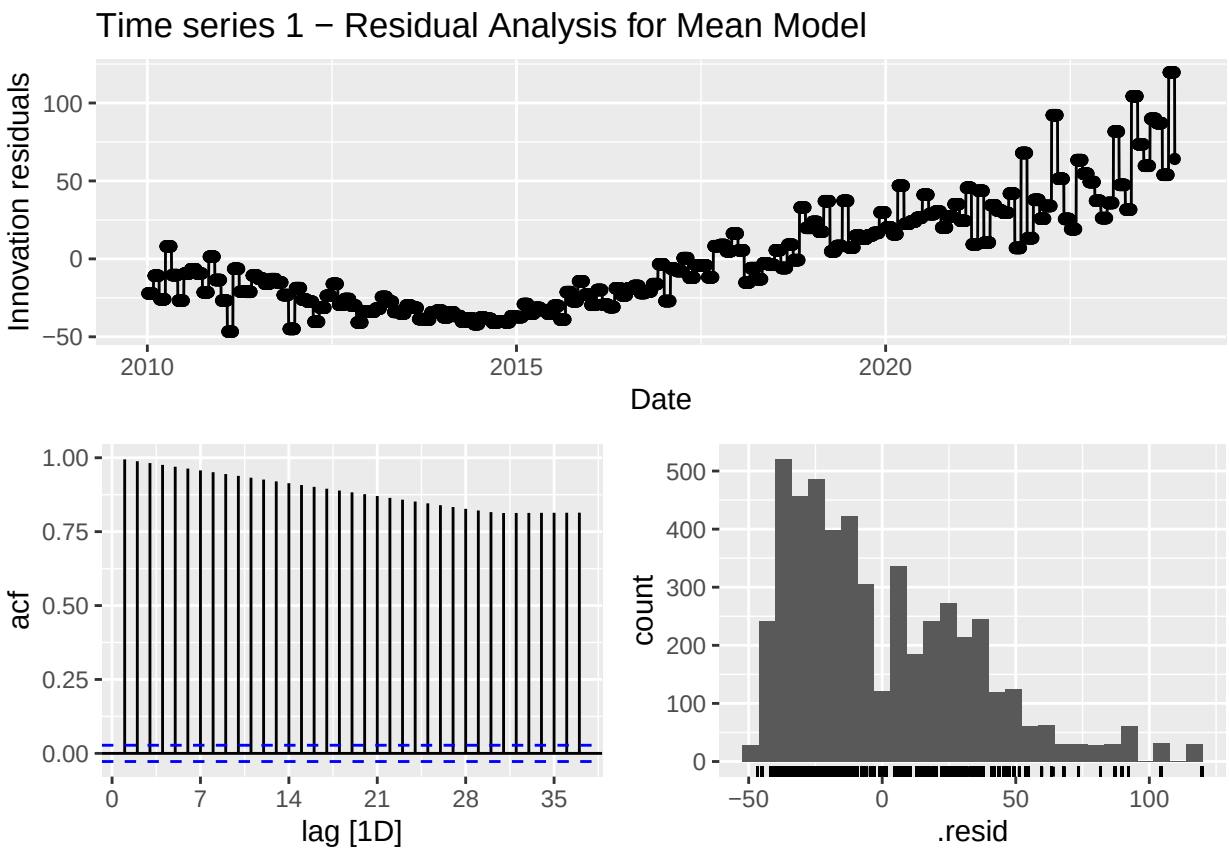
The correlogram for the time series 2 (monthly) shows significant autocorrelations at lags 1 to 4 and 16 to 22, with 11 out of 48 lags being significant. This indicates clear evidence of dependencies within the time series. However, while dependencies exist, the lack of consistent spikes suggests the seasonal pattern is not as strongly evident as in the subseries plot of time series 2 (monthly). The declining autocorrelations beyond lag 22 further suggest diminishing influence of earlier values on later ones.

IV. Residual analysis

4. *Perform residual analysis. Specifically:*
   i. *Estimate the 4 simple models: naïve, mean, seasonal naïve, drift.*
   ii. *Perform residual analysis, commenting on the presence of the 4 properties.*

```
mod_m_dataset_1 <- ts_dataset_1_fill %>%
  model(MEAN(Value))
mod_n_dataset_1 <- ts_dataset_1_fill %>%
  model(NAIVE(Value))
mod_s_dataset_1 <- ts_dataset_1_fill %>%
  model(SNAIVE(Value))
mod_d_dataset_1 <- ts_dataset_1_fill %>%
  model(RW(Value ~ drift()))
```

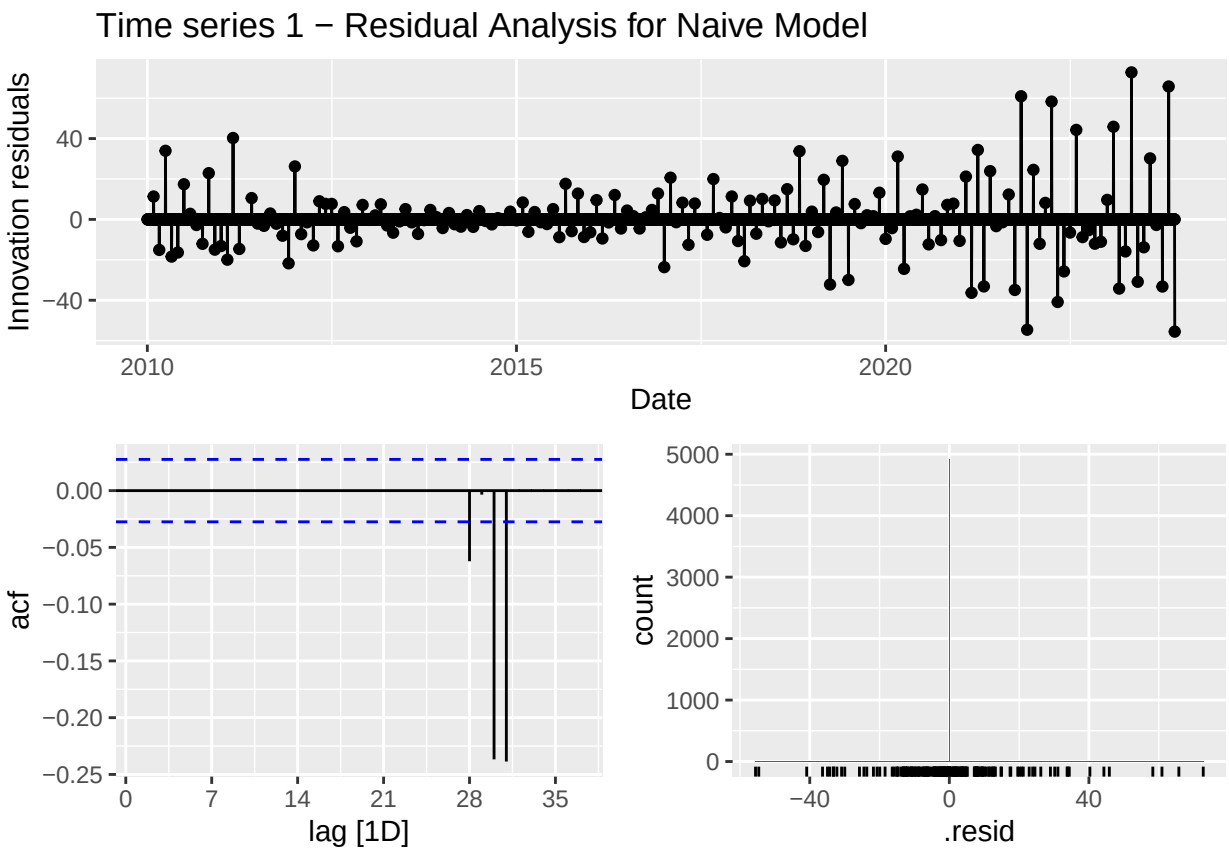
```
gg_tsresiduals(mod_m_dataset_1) +
  labs(title = "Time series 1 - Residual Analysis for Mean Model ")
```



The residual analysis for the mean model of time series 1 highlights major issues. Residuals stray far from zero over time, showing the model's bias and failure to capture key patterns. Strong significant autocorrelations at all lags suggest it doesn't handle dependencies well. The increasing variance and skewed histogram point to deviations from normality and inconsistent variance. Overall, the mean model struggles to reflect the underlying trends and requires a better approach.

```
gg_tsresiduals(mod_n_dataset_1) +
  labs(title = "Time series 1 - Residual Analysis for Naive Model ")
```

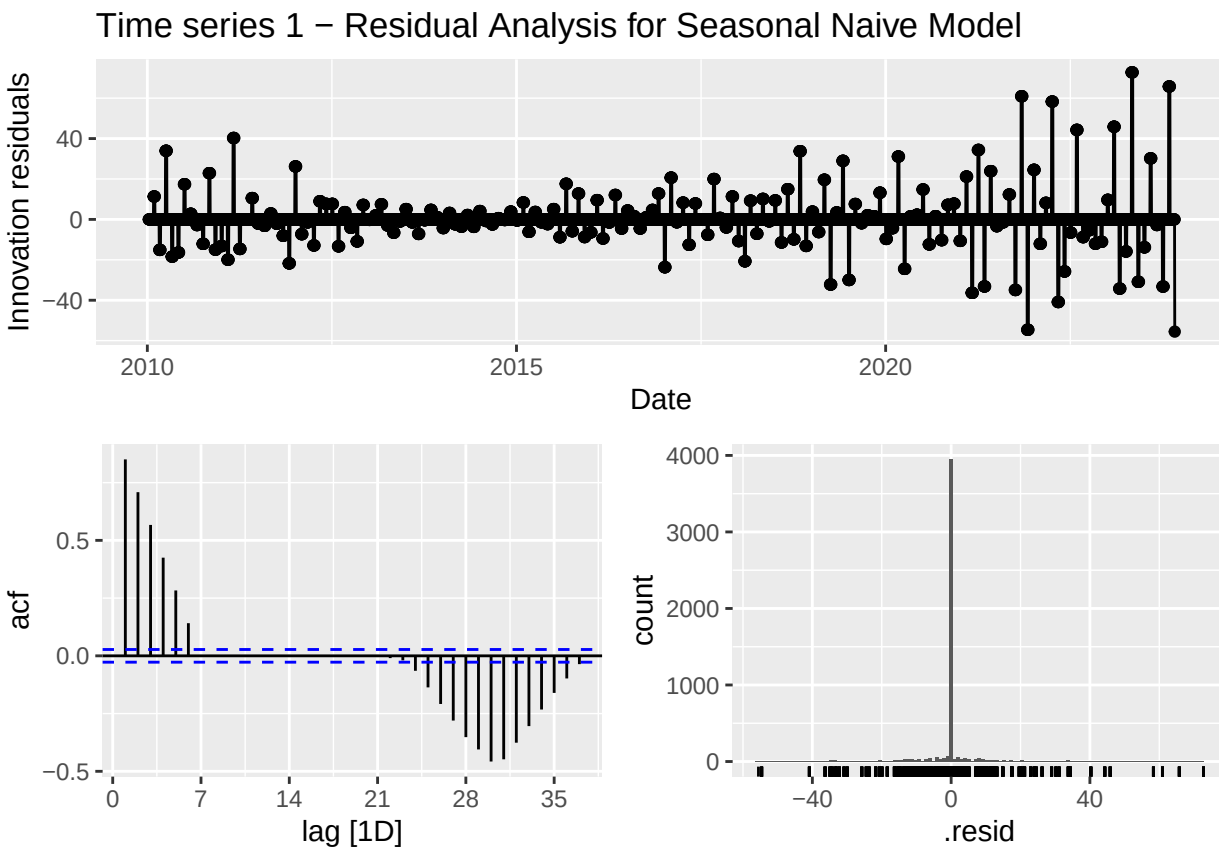
```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_bin()`).
```



The residuals for the SNaive model are mostly centered around zero, showing unbiased predictions. Residual variance looks consistent, suggesting the model maintains homoscedasticity. The ACF plot shows minimal autocorrelation with a few major spikes at later lags, indicating the model mostly captures the seasonal pattern well. The histogram shows most residuals near zero, though there are a few outliers. Overall, the SNaive model handles the seasonal pattern effectively, with some room for improvement in managing outliers.

```
gg_tsresiduals(mod_s_dataset_1) +
  labs(title = "Time series 1 - Residual Analysis for Seasonal Naive Model ")
```

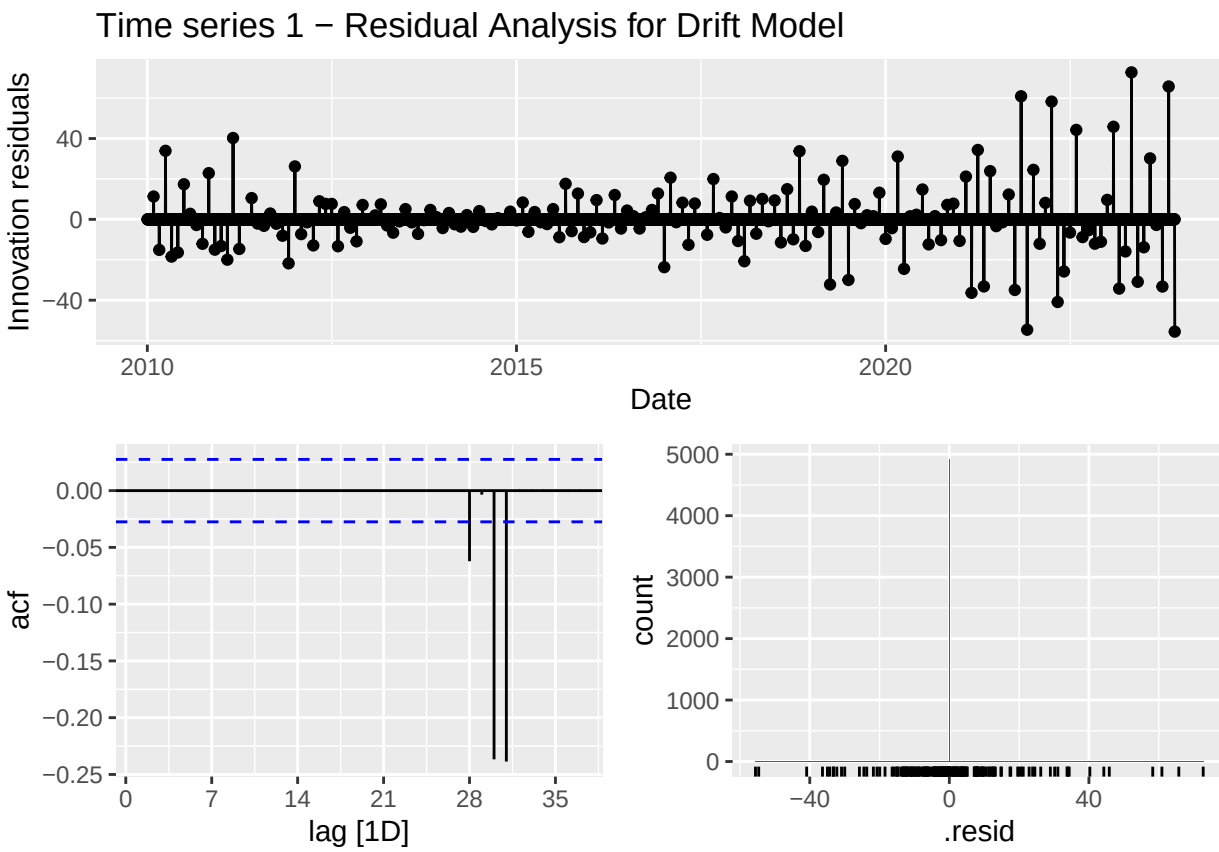
```
## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 7 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



The residuals for this model are centered around zero, with no evident bias over time, as seen in the time series plot and histogram. The variance appears stable across the observed period, indicating homoscedasticity. The autocorrelation function (ACF) plot shows significant spikes in early lages and later lags as well, meaning seasonlity is not captured very well in the model. Also, the histogram highlights a concentration of residuals near zero, with a few extreme outliers. Overall, while the model performs well, addressing lower level seasonlity and the outliers could improve robustness.

```
gg_tsresiduals(mod_d_dataset_1) +
  labs(title = "Time series 1 - Residual Analysis for Drift Model ")
```

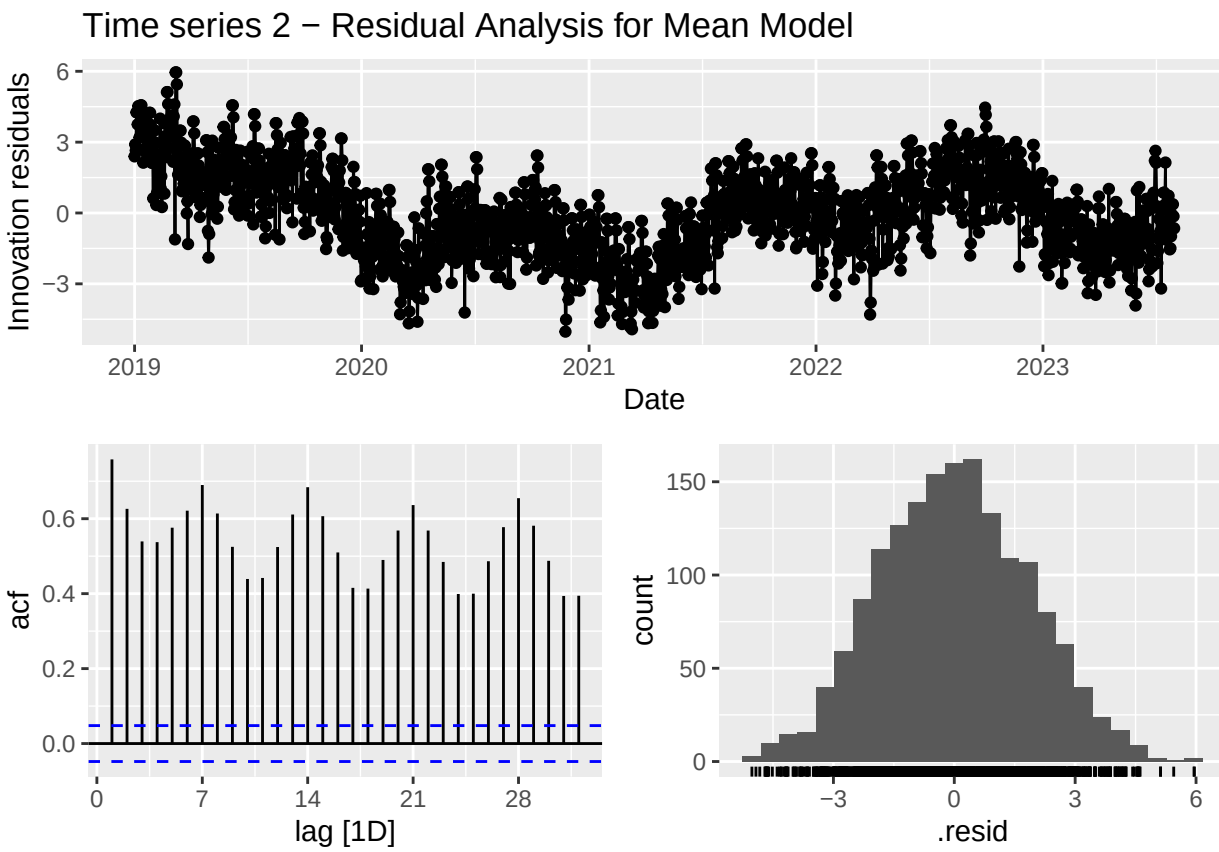
```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_bin()`).
```



The residual analysis for this model indicates satisfactory performance in several aspects. The residuals are centered around zero and show minimal bias over time, with the histogram indicating a very spaced out distribution, not necessarily the bell shaped normal distribution that is aimed for. The ACF plot reveals negligible autocorrelation, confirming that the model effectively captures time dependencies. Finally, the variance of residuals is consistent over the observed period, satisfying the assumption of homoscedasticity.

```
mod_m_dataset_2 <- ts_dataset_2 %>%
  model(MEAN(Value))
mod_s_dataset_2 <- ts_dataset_2 %>%
  model(SNAIVE(Value))
mod_n_dataset_2 <- ts_dataset_2 %>%
  model(NAIVE(Value))
mod_d_dataset_2 <- ts_dataset_2 %>%
  model(RW(Value ~ drift()))
```

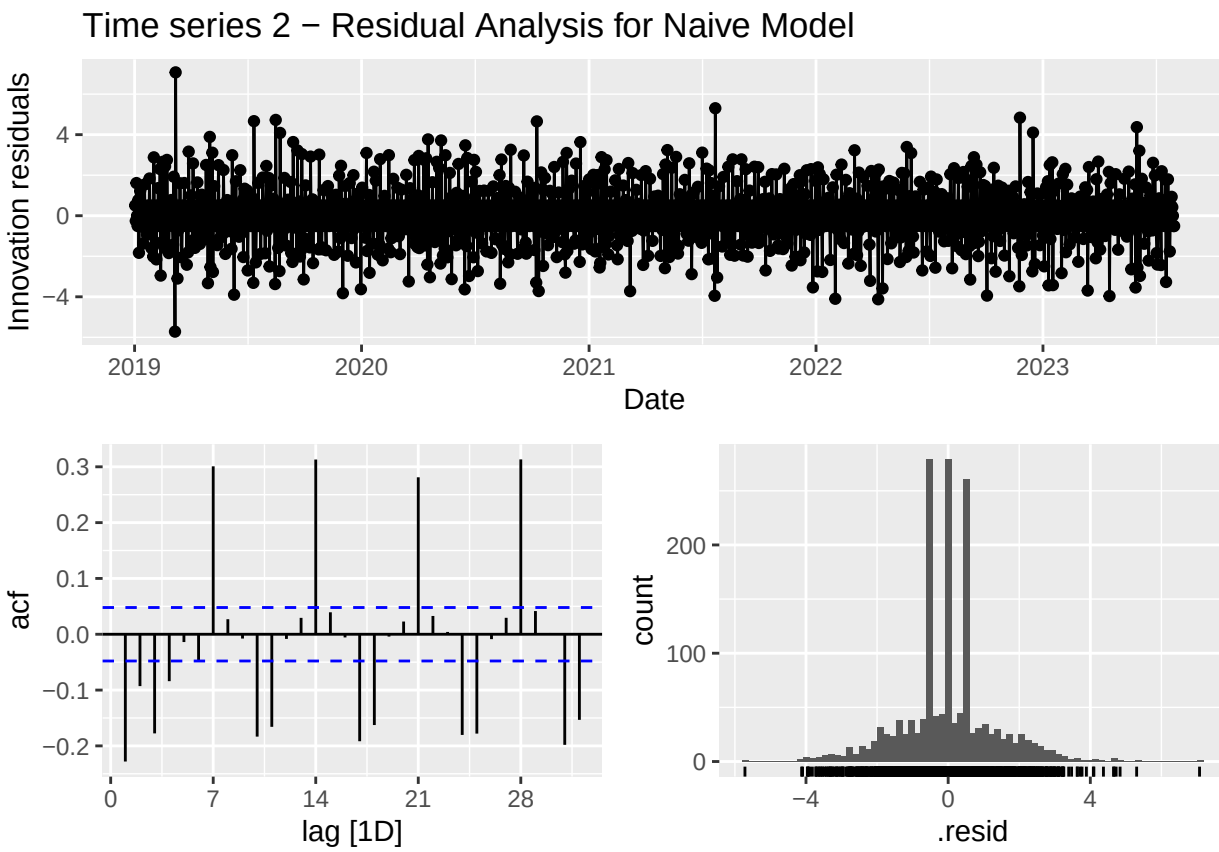
```
gg_tsresiduals(mod_m_dataset_2) +
  labs(title = "Time series 2 - Residual Analysis for Mean Model ")
```



The residuals are centered around zero, indicating unbiased predictions. However, significant spikes and patterns in the ACF plot at lags 7, 14, 21 and 28 suggest the model fails to fully capture certain time dependencies or seasonal patterns. The residual variance is consistent, meeting the assumption of homoscedasticity. The histogram shows an almost perfect bell shaped normal distribution. Thus, to improve this model, seasonality showcased by the ACF of the resiudals has to be addressed to improve the model.

```
gg_tsresiduals(mod_n_dataset_2) +
  labs(title = "Time series 2 - Residual Analysis for Naive Model ")
```

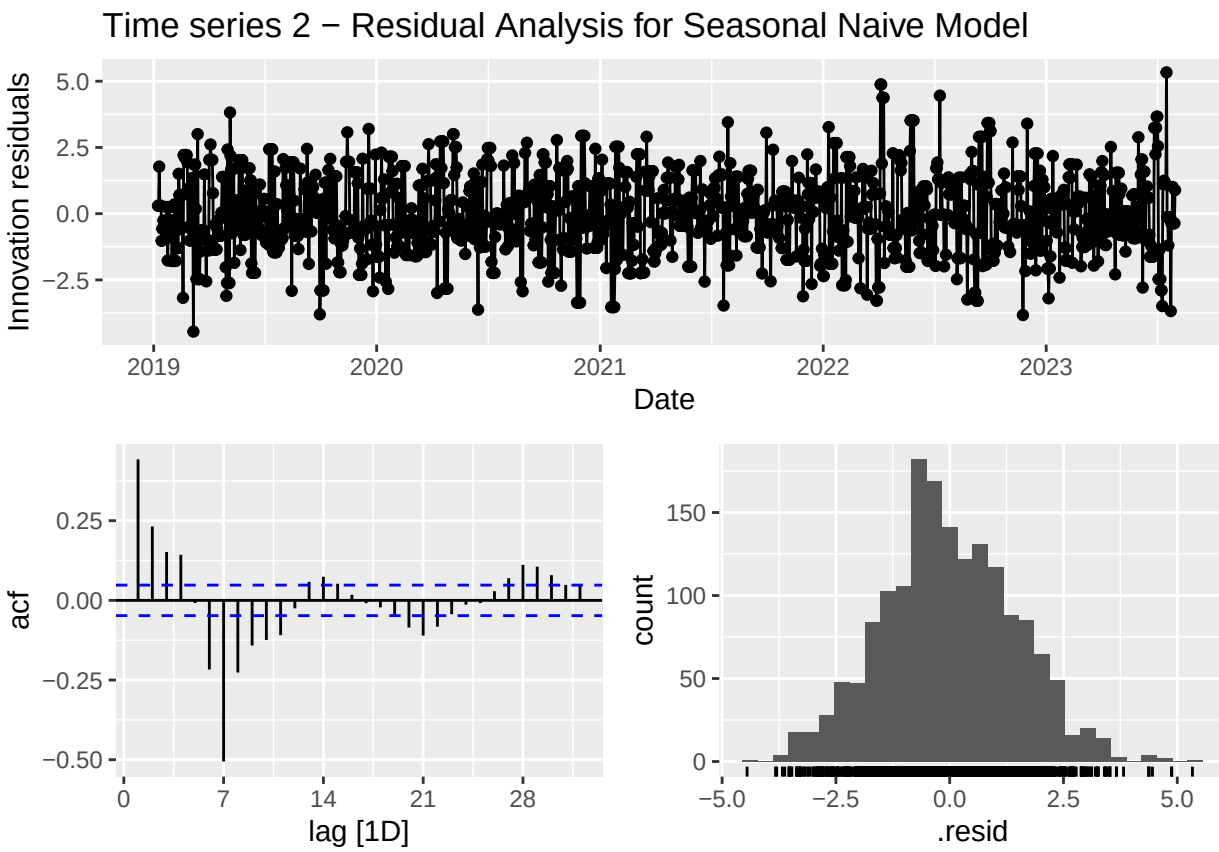
```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_bin()`).
```



For the naive model on time series 2, residuals are centered around zero, showing unbiased predictions. The ACF has significant spikes, indicating missed time dependencies. The variance is stable, meeting homoscedasticity. The multimodal histogram suggests the presence of systematic patterns or cycles not captured by the model.

```
gg_tsresiduals(mod_s_dataset_2) +
  labs(title = "Time series 2 - Residual Analysis for Seasonal Naive Model ")
```

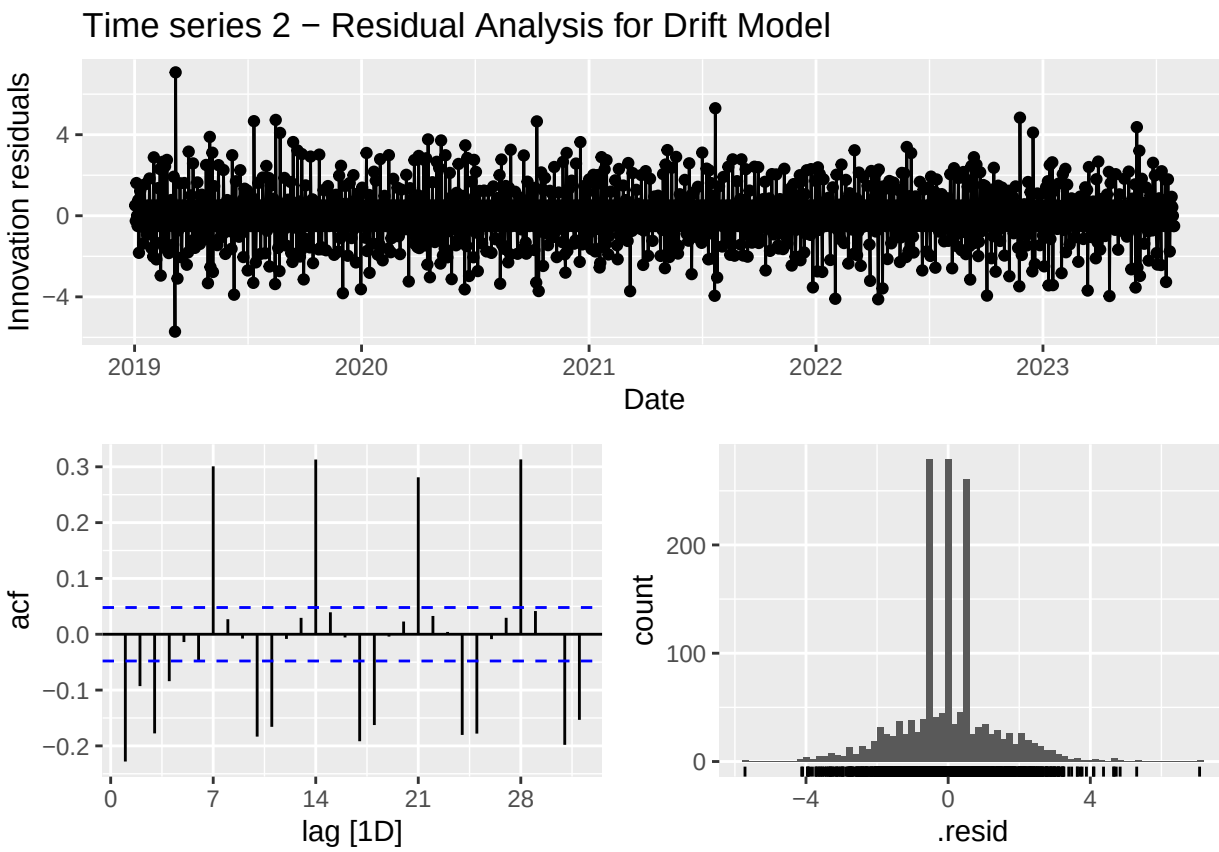
```
## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 7 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



The residuals are centered around zero in both the histogram and time series plot, indicating unbiased predictions. However, significant spikes in the ACF plot at multiple lags suggest that the model has not fully captured important time dependencies or seasonal patterns. The variance of residuals appears relatively consistent over time, satisfying the homoscedasticity assumption. While the residuals meet the mean zero and homoscedasticity criteria, the significant autocorrelation suggests that the model could be improved to better account for time dependencies or seasonality.

```
gg_tsresiduals(mod_d_dataset_2) +
  labs(title = "Time series 2 - Residual Analysis for Drift Model ")
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_bin()`).
```



For the drift model on time series 2, residuals are centered around zero, showing unbiased predictions. The ACF has significant spikes, indicating missed time dependencies. The variance is stable, meeting homoscedasticity. The multimodal histogram suggests the presence of systematic patterns or cycles not captured by the model.

V. Forecasting

5. *Now, make forecasts:*

   i. *Create* `tsibbles` `train` *and* `test`*:*

      - *For* `data1`*, allocate the initial 80% of observations (the first 80% of observations chronologically speaking) to training, and the rest to testing. If 80% does not correspond to an integer, round the number to be close to 80%).*
      - *For* `data2`*, allocate the last 6 complete months to testing.*

   ii. *Estimate the simple models on training data only and make forecasts over the appropriate horizon.*

   iii. *Make a graph of your forecasts, also including the training and testing observations. Produce 2 plots: one without probabilistic predictions and one with the 90% prediction interval.*

   iv. *Compute accuracy metrics.*

   v. *Comment on the visualization of the forecasts and on their accuracy. What do we learn? Is there a specification that clearly performs better? Which one is it and why do you think that is the case?*

DATA SET 1

```r
# Allocate the initial 80% of data to training and the remaining to testing
train_ts_dataset_1 <- ts_dataset_1_fill %>%
  slice(1:floor(0.8 * nrow(ts_dataset_1_fill)))
test_ts_dataset_1 <- ts_dataset_1_fill %>%
  slice((floor(0.8 * nrow(ts_dataset_1_fill)) + 1):nrow(ts_dataset_1_fill))
```

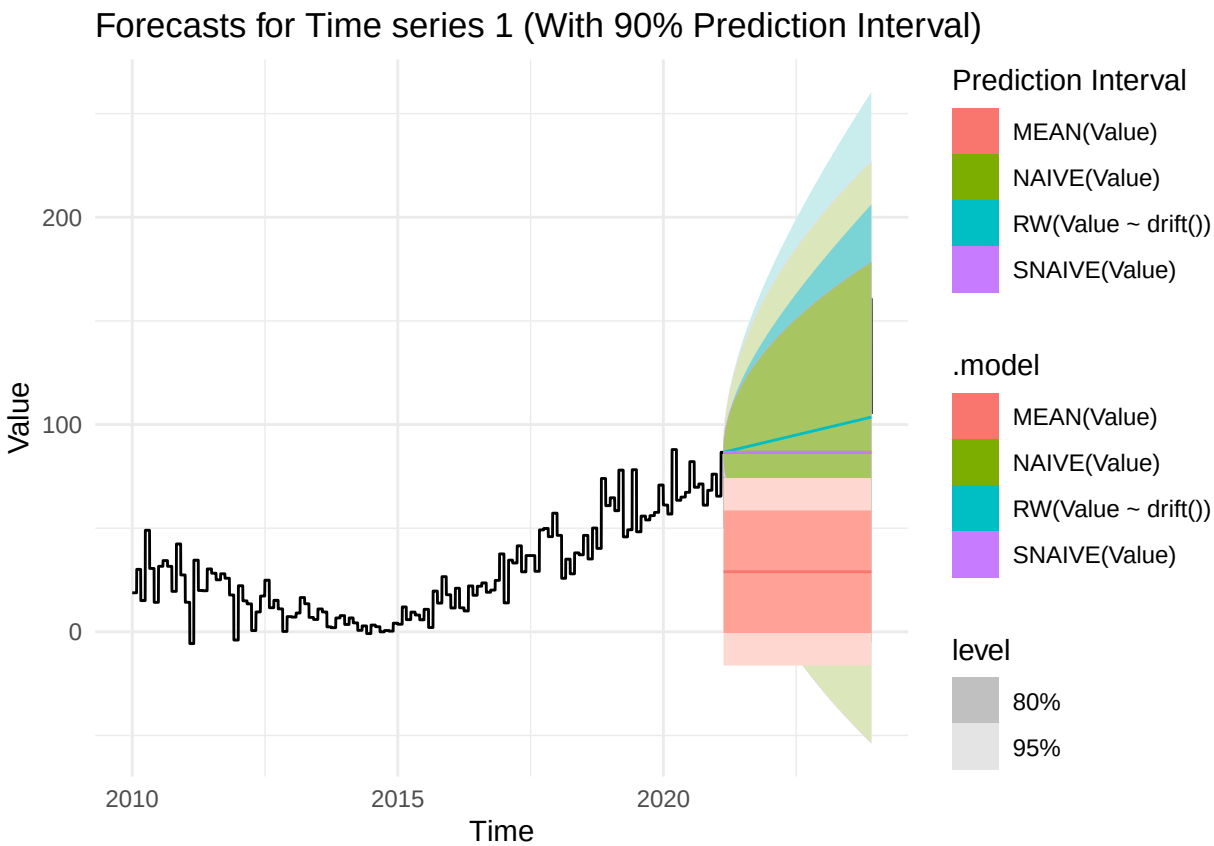
```r
# Fit the models using the training dataset
mod_m_train <- train_ts_dataset_1 %>%
  model(MEAN(Value))
mod_n_train <- train_ts_dataset_1 %>%
  model(NAIVE(Value))
mod_s_train <- train_ts_dataset_1 %>%
  model(SNAIVE(Value))
mod_d_train <- train_ts_dataset_1 %>%
  model(RW(Value ~ drift()))
```

```r
# Create forecasts for each model using the training data
forecasts <- bind_rows(
  mod_m_train %>% forecast(h = nrow(test_ts_dataset_1)),
  mod_n_train %>% forecast(h = nrow(test_ts_dataset_1)),
  mod_s_train %>% forecast(h = nrow(test_ts_dataset_1)),
  mod_d_train %>% forecast(h = nrow(test_ts_dataset_1))
)
```

*The graph below combines probabilistic predictions with its 90% prediction intervals.

```r
autoplot(train_ts_dataset_1, Value) +
  autolayer(test_ts_dataset_1, Value, series = "Testing Data") +
  autolayer(forecasts, series = "Forecasts") +
  labs(
    title = "Forecasts for Time series 1 (With 90% Prediction Interval)",
    y = "Value",
    x = "Time"
  ) +
  theme_minimal() +
  guides(fill = guide_legend(title = "Prediction Interval"))
```

```
## Warning in geom_line(eval_tidy(expr(aes(!!!aes_spec))), data = object, ..., :
## Ignoring unknown parameters: `series`

## Warning in ggdist::geom_lineribbon(without(intvl_mapping, "colour_ramp"), :
## Ignoring unknown parameters: `series`

## Warning in geom_line(mapping = without(mapping, "shape"), data =
## unpack_data(object[single_row[["FALSE"]], : Ignoring unknown parameters:
## `series`
```




```r
# Calculate accuracy metrics
accuracy_metrics <- forecasts %>%
  accuracy(test_ts_dataset_1)

print(accuracy_metrics)
```

```
## # A tibble: 4 x 10
##   .model              .type    ME  RMSE   MAE    MPE  MAPE  MASE RMSSE  ACF1
##   <chr>               <chr> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 MEAN(Value)         Test  60.0   66.2  60.0  64.2   64.2   NaN   NaN 0.974
## 2 NAIVE(Value)        Test   2.37  28.0  22.1  -6.87  26.1   NaN   NaN 0.974
## 3 RW(Value ~ drift()) Test  -6.12  25.8  21.6 -16.2   27.7   NaN   NaN 0.968
## 4 SNAIVE(Value)       Test   2.37  28.0  22.1  -6.87  26.1   NaN   NaN 0.974
```

The accuracy metrics indicate that the Mean model performs the worst, with the highest RMSE (66.16) and MAE (60.00), suggesting poor predictive accuracy. The Naive and Seasonal Naive models perform similarly, with lower RMSE (27.98) and MAE (22.10), and a consistent MAPE of 26.10%. The Random Walk with Drift model shows slightly better RMSE (25.82) and MAE (21.58), indicating it provides the most accurate forecasts among the models. However, all models exhibit high autocorrelation in residuals (ACF1 ~ 0.97), implying potential room for improvement in capturing dependencies in the series.

DATA SET 2

```r
# Split the data into training and testing
train_ts_dataset_2 <- ts_dataset_2 %>%
  filter_index(. ~ "2023-01")  # Training: up to and including January 2023

test_ts_dataset_2 <- ts_dataset_2 %>%
```

```r
  filter_index("2023-02" ~ .)  # Testing: February 2023 onwards
```

```r
# Estimate simple models on the training data
mod_m_train <- train_ts_dataset_2 %>%
  model(Mean = MEAN(Value),
        Naive = NAIVE(Value),
        SNaive = SNAIVE(Value),
        Drift = RW(Value ~ drift()))
```

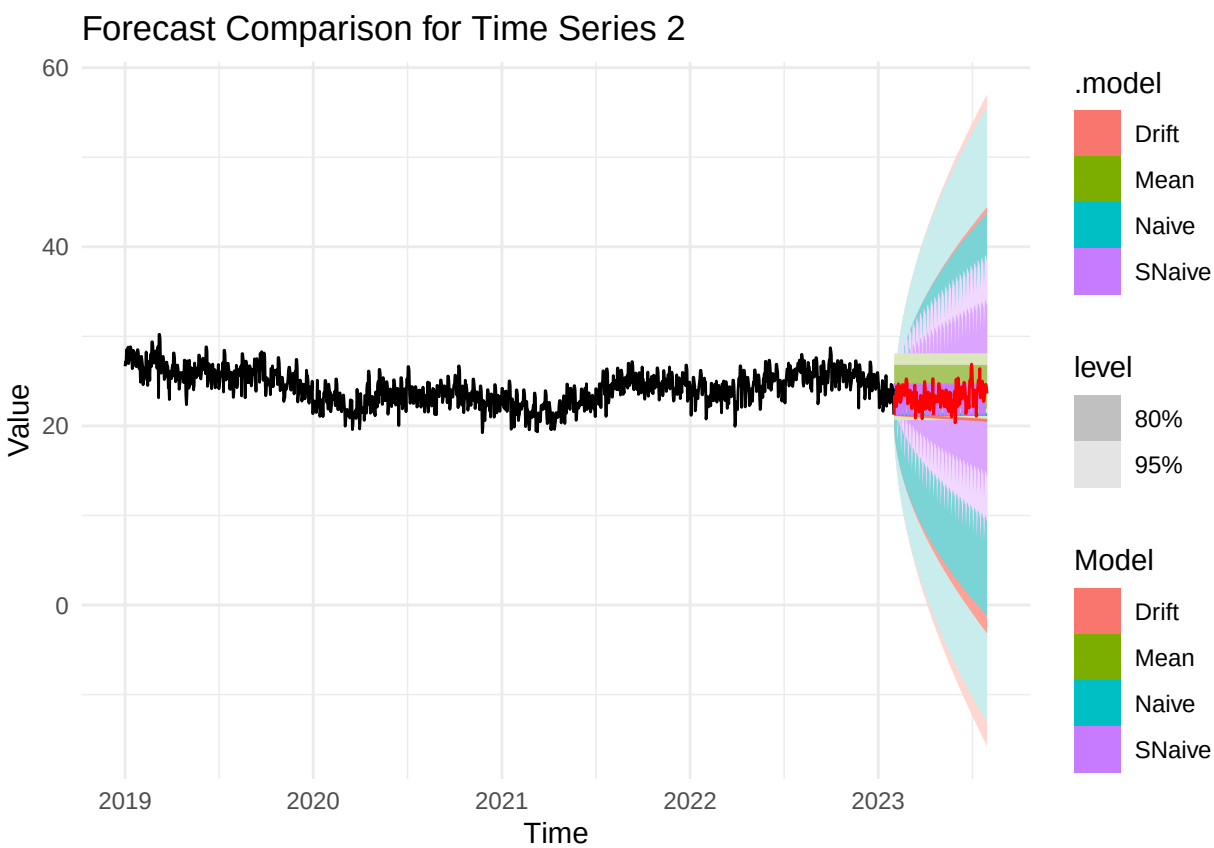
```r
# Generate forecasts for the testing period
forecasts <- mod_m_train %>%
  forecast(h = nrow(test_ts_dataset_2))
```

*The graph below combines probabilistic predictions with its 90% prediction intervals.

```r
# Plot forecasts with training and testing data
autoplot(forecasts, train_ts_dataset_2) +
  autolayer(test_ts_dataset_2, color = "red", series = "Test Data") +  # Ensure 'series' argument is us
  labs(title = "Forecast Comparison for Time Series 2",
       x = "Time",
       y = "Value",
       color = "Model") +  # Only one color legend
  theme_minimal()
```

```
## Plot variable not specified, automatically selected `.vars = Value`
```

```
## Warning in geom_line(eval_tidy(expr(aes(!!!aes_spec))), data = object, ..., :
## Ignoring unknown parameters: `series`
```



```r
# Compute accuracy metrics
accuracy_metrics <- forecasts %>%
  accuracy(test_ts_dataset_2)

accuracy_metrics
```

```
## # A tibble: 4 x 10
##   .model .type     ME  RMSE   MAE   MPE  MAPE  MASE RMSSE  ACF1
##   <chr>  <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Drift  Test   2.40   2.69 2.41  10.1  10.1    NaN   NaN 0.475
## 2 Mean   Test  -1.02   1.55 1.27  -4.65  5.61   NaN   NaN 0.438
## 3 Naive  Test   2.07   2.38 2.10   8.66  8.80   NaN   NaN 0.438
## 4 SNaive Test  -0.355  1.22 0.984 -1.66  4.27   NaN   NaN 0.265
```

The accuracy metrics for time series 2 indicate that the Seasonal Naive (SNaive) model performs the best, with the lowest RMSE (1.22), MAE (0.98), and MAPE (4.27%), suggesting it effectively captures the seasonal structure of the data. The Mean model follows with moderate RMSE (1.55) and MAE (1.27) but a higher MAPE (5.61%), indicating less precise forecasts. The Naive model performs poorly with higher RMSE (2.38) and MAPE (8.80%), and the Drift model shows the highest errors, including the largest MAPE (10.12%). Residual autocorrelations (ACF1) are lowest for SNaive (0.26), confirming its better residual fit compared to others.

Even though the visual representation may hint towards the drift model being the most accurate, the accuracy measures highlight that simply looking at the model visually can be deceiving.